

DevOps culture through a failed release

Traditional delivery, Agile roots, continuous delivery, and the role of cloud

One Friday release, four late surprises

4:30 PM



ZIP file arrives

Operations sees the change for the first time.

6:00 PM



Test setup differs

QA did not use the production configuration.

9:10 PM



Database connection fails

A required environment variable is missing.

10:20 PM



Rollback completes

The previous version remains in service.

Evidence in the failed release

What facts do you notice?

- 1 Operations sees the release late.
- 2 QA and production use different configurations.
- 3 A manual step controls a critical setting.
- 4 The defect reaches production before anyone detects it.

Which fact would you investigate first, and why?

Learning outcomes

- 1 Compare traditional handoffs with shared DevOps ownership**

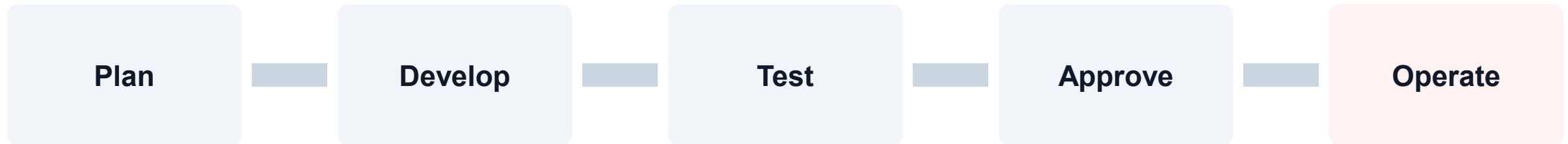
- 2 Explain how DevOps grew from Agile collaboration**

- 3 Judge whether speed produces useful delivery evidence**

- 4 Connect cloud capabilities to earlier testing and feedback**

Syllabus focus: Traditional development vs DevOps culture; role of cloud in DevOps

Traditional delivery depends on handoffs



Each boundary adds a queue, a translation, and a chance to lose context.

In the case, code, test evidence, deployment instructions, and configuration move separately.

Departmental success can hide service failure

Development

Code was complete

release handed off

QA

Tests were executed

different configuration

Operations

Procedure was followed

late discovery

Students

Portal update was delayed

service outcome

The service outcome reveals whether the whole delivery system succeeded.

DevOps grew from a collaboration problem

2001 The Agile Manifesto emphasizes individuals and interactions.

2009 Patrick Debois organized DevOpsDays in Ghent after efforts to close the development-operations divide.

DevOps The practice combines technical delivery work with cultural change across disciplines.

Shared responsibility for one service

Development, testing, operations, security, and other contributors share information and responsibility for delivering and running one service.

Specialists keep their expertise.

The work crosses role boundaries earlier.

The shared goal is useful change with reliable operation.

Shared workflow keeps context visible

SEPARATE SYSTEMS

Development

Feature tracker

QA

Bug tracker

Operations

Deployment requests

SHARED WORKFLOW

One change record, role-specific views, and evidence visible to every contributor

Technology should improve human interaction

Visible work

A shared record shows the change, owner, status, and evidence.

Repeatable checks

Automation removes routine manual work and returns results sooner.

Time for judgment

People can spend more time on design, risk, and user needs.

A tool earns its place when it reduces friction or improves feedback across roles.

Useful speed reduces waiting and advances value

Speed without value

Faster approval of a large batch

Faster execution of a fragile manual process

More output that users do not need

Speed toward evidence

Earlier stakeholder review

Shorter build, test, and deploy waits

Faster learning from a small change

Which waiting time in the Friday case delays useful evidence?

Smaller changes make feedback easier to use

Eight-week release

Many features

Many configuration changes

Many possible causes

Small release

Fewer changes

Faster evidence

Simpler recovery

A small change narrows the investigation when something fails.

Agile work runs at several cadences



Short technical loops support the longer planning cycles.

DevOps supports every cadence with delivery evidence

Minutes

Edit, build, and test

A developer learns whether a change works.

Day

Operational and Kanban flow

A team sees blocked work and restores flow.

Weeks

Scrum sprint

Stakeholders inspect a working increment.

Months

Program and portfolio decisions

Leaders compare outcomes with investment.

An eight-week handoff leaves the shortest feedback loops unused.

Cargo cult DevOps misses the purpose

Daily stand-up

People report status, but teams still hide work from one another.

Automation tool

Only one team understands it, so the same boundary remains.

Deployment pipeline

It moves work faster for one group while users wait just as long.

Goal test: who receives useful evidence sooner, and what decision improves?

Practices need a clear delivery purpose

Create a pipeline

Return reliable test evidence before production

Hold a retrospective

Turn the repeated failure into a preventive change

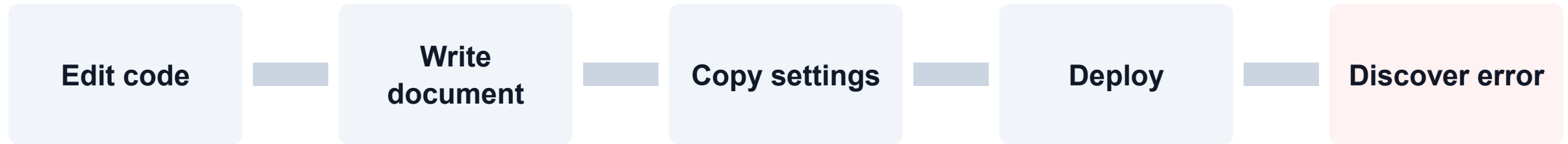
Adopt a shared tracker

Make change context visible across roles

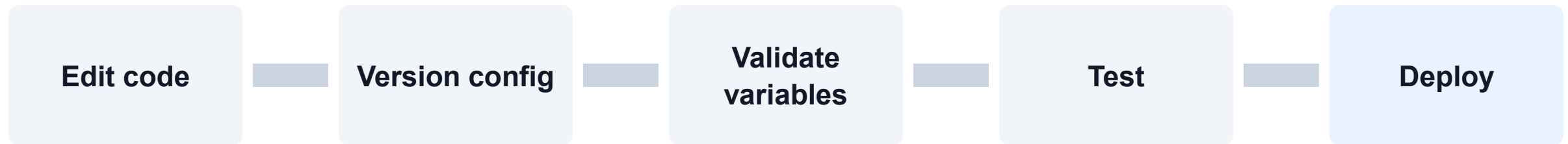
The practice matters when it changes feedback, behavior, or the service outcome.

Automation catches the missing variable earlier

CURRENT PATH

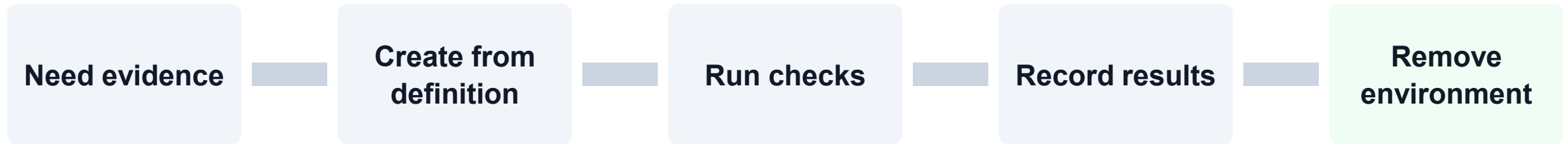


IMPROVED PATH



The check returns a clear result while the change is still easy to correct.

Cloud can shorten the path to a realistic test



DEVOPS CONNECTION On-demand, API-accessible infrastructure makes repeatable environments practical.

The team still needs access rules, cost limits, validation, and cleanup.

Cloud capability needs engineering discipline

CAPABILITY

On-demand resources

POSSIBLE BENEFIT

Earlier testing

RISK TO CONTROL

Unused resources and cost

Automation through APIs

Repeatable setup

Fast, repeated mistakes

Elastic capacity

Load testing

Consumption without limits

Measured service

Usage visibility

Data with no action

Cloud amplifies the delivery practices that a team already uses.

DevOps and ITIL can serve different change needs

Small, frequent changes

- Automated checks
- Traceable evidence
- Repeatable deployment
- Fast recovery

Large legacy changes

- More dependencies
- Formal coordination
- Detailed change control
- Higher planning cost

Governance can use DevOps evidence while matching controls to change risk.

Evidence for the redesigned release

Lead time

How long until a change is ready for deployment?

Delivery frequency

Can the team deliver smaller changes more often?

Failed changes

How often does a deployment need rollback or urgent correction?

Recovery time

How quickly does the service return after an incident?

Repeated failures

Did the same preventable problem happen again?

Use measures to choose the next improvement, not to rank individuals.

Team worksheet

Eight-week releases. ZIP-file handoff. Manual production configuration. Late QA environment. Saturday deployment. A missing variable causes a 70-minute rollback. A similar failure happened before.

1

Record facts

No solutions yet

2

Find the delay

Where is evidence late?

3

Redesign the system

People, process, technology

4

Choose evidence

How will you know?

Use Section 12 of your student notes. Work in teams of 3 or 4.

Self-check

- 1 Why can traditional handoffs hide a service-level failure?**

- 2 How did Agile ideas influence the DevOps movement?**

- 3 What distinguishes useful speed from faster waste?**

- 4 How can a cloud capability shorten feedback, and what risk remains?**

- 5 How can DevOps evidence support formal change control?**

Chapter 1 reading guide

Printed pages 1-3

DevOps origins, collaboration across disciplines, and the role of tools

Printed pages 3-5

Business-value speed and the Agile wheel of wheels

Printed pages 6-7

Cargo cult Agile and the goal test for practices

Printed pages 7-8

DevOps and ITIL in larger organizations

[Books/practical-devops.pdf](#). Chapter 1 is on PDF pages 20-27.

Which responsibilities change with IaaS, PaaS, and SaaS?

EXIT REFLECTION

Name one traditional delivery constraint and one DevOps response supported by Chapter 1.